

Building Intelligent Flutter Applications: From On-Device ML (ML Kit) to Cloud-Native Agents (Firebase AI & Genkit)

The New Frontier: Why Flutter and AI?

This presentation serves as a technical deep-dive into the architectural decisions and practical implementations required to build intelligent, AI-first applications using the Flutter framework. We will progress from foundational on-device machine learning to advanced, cloud-based agentic workflows.

The 2025 Mandate: "AI-First" Applications

The software development paradigm has fundamentally shifted. The "mobile-first" approach is no longer the competitive benchmark; it has been superseded by an "AI-first" mandate. The global market for Artificial Intelligence reflects this, with projections indicating it will surpass \$826 billion by 2030.¹

This trend is driven by a definitive change in user expectations. Users no longer demand applications that are merely *reactive*; they expect applications that are *predictive*, responsive, and intuitive.¹ This evolution brings us to the concept of "Flutter AI".¹

"Flutter AI" is not a specific product. It is an architectural philosophy: the fusion of Flutter's unparalleled UI capabilities and cross-platform flexibility with the power of machine learning, natural language processing (NLP), and intelligent automation.¹ The objective is no longer to simply "add AI" to an existing application. Instead, the goal is to build applications where AI is the core, and Flutter is the vessel to deliver that intelligence to any screen.

The Core Architectural Decision

Before implementing any AI feature, developers face a critical architectural decision: where will the application's "brain" reside? There are three primary models, each with distinct

trade-offs.

1. **On-Device AI:** The machine learning model runs entirely on the user's device. This approach is characterized by high speed, strong data privacy, and offline functionality.³ This will be explored using **Google ML Kit**.
2. **Cloud AI:** The model resides on a server. The application sends requests (prompts) and receives responses. This provides access to near-unlimited computational power, enabling the use of massive, complex models.³ This will be explored using **Firestore AI Logic**.
3. **Hybrid AI:** This is a sophisticated architecture that balances the two. A lightweight, on-device model might be used for quick, private, or real-time tasks (e.g., "hotword" detection), which then triggers a more powerful cloud-based API for heavy-lifting (e.g., processing the full user query).⁴

This lecture will analyze the first two architectures in detail, providing the foundation to design effective hybrid solutions.

On-Device Intelligence: Google ML Kit for Flutter

What is On-Device AI?

On-device AI, or "edge inference," involves performing all machine learning processing locally on the user's hardware. No data is sent to a remote server.⁷

This architecture provides three primary, powerful advantages:

- **Speed:** Processing happens locally, eliminating network latency. This is essential for real-time use cases, such as processing a live camera feed to detect faces or scan barcodes.⁷
- **Privacy:** This is arguably the greatest strength of on-device AI. Sensitive user data, such as personal images, documents, or message histories, never leaves the device.³ This simplifies compliance with data privacy regulations and builds user trust.
- **Offline Functionality:** Because the model is bundled with the app, its features work without an internet connection.³ This is a critical requirement for applications intended for areas with poor connectivity or for core functionality that must be infallible.

Introduction to Google ML Kit for Flutter

The primary toolset for on-device AI in the Flutter ecosystem is the suite of Google ML Kit plugins.⁹ ML Kit is a mobile-optimized SDK that bundles Google's machine learning expertise into a series of easy-to-use APIs for common mobile use cases, such as text recognition, face

detection, and image labeling.⁷

However, for a university-level and production-focused audience, several critical clarifications about the *Flutter* implementation are necessary.

- **Community Maintained:** The Flutter plugins, such as `google_mlkit_face_detection`, are **not** officially sponsored or maintained by Google. They are high-quality, community-driven wrappers.¹¹ This has significant implications for long-term maintenance and support.
- **Platform Channels:** These plugins function by using Flutter's Platform Channel mechanism to call the *native* Google ML Kit APIs on iOS and Android.¹¹ This architecture is an abstraction layer, and developers may encounter native-specific issues.¹³
- **Platform Support:** ML Kit is designed exclusively for mobile platforms (iOS and Android). It does **not** support web.¹¹
- **Dependency Management:** Developers must **not** use the "umbrella" package `google_ml_kit` in their `pubspec.yaml`. This package includes *all* features and their respective dependencies, which will "significantly increasing the size of your app".⁹ The correct, production-ready practice is to add only the specific plugins for the features being consumed (e.g., `flutter pub add google_mlkit_barcode_scanning`).⁹

ML Kit Vision APIs

ML Kit's most robust features are its Vision APIs, which provide a wide range of image and video analysis tools.⁷ Because these are wrappers around native APIs, it is essential to verify platform-specific support. For example, some beta features like 'Face Mesh Detection' and 'Document Scanner' are only available on Android.⁹

The table below summarizes the most common Vision APIs and their corresponding Flutter plugins.⁹

Feature	Plugin	Android	iOS
Barcode Scanning	<code>google_mlkit_barcode_scanning</code>	✓	✓
Face Detection	<code>google_mlkit_face_detection</code>	✓	✓
Text Recognition v2	<code>google_mlkit_text_recognition</code>	✓	✓
Image Labeling	<code>google_mlkit_image_labeling</code>	✓	✓
Object Detection	<code>google_mlkit_object_detection</code>	✓	✓
Pose Detection (Beta)	<code>google_mlkit_pose_detection</code>	✓	✓
Face Mesh Detection	<code>google_mlkit_face_mesh</code>	✓	✗

(Beta)	h_detection		
Document Scanner (Beta)	google_mlkit_document_scanner	✓	✗

ML Kit NLP APIs

Alongside its Vision capabilities, ML Kit provides several powerful on-device Natural Language Processing (NLP) APIs. These APIs perform complex language tasks while preserving user privacy by keeping conversation data on the device.

A prominent example is the **Smart Reply API**.¹⁴ This API analyzes a conversation history locally and suggests up to three context-aware replies. It is optimized for casual English conversations and is designed to help users respond quickly, especially on devices with limited input.¹⁴

The table below outlines the available on-device NLP APIs.⁹

Feature	Plugin	Android	iOS
Language Identification	google_mlkit_language_id	✓	✓
On-Device Translation	google_mlkit_translation	✓	✓
Smart Reply	google_mlkit_smart_reply	✓	✓
Entity Extraction (Beta)	google_mlkit_entity_extraction	✓	✓

Code Deep Dive: google_mlkit_image_labeling

The implementation workflow for most ML Kit plugins is consistent and straightforward. The following example demonstrates the use of google_mlkit_image_labeling.¹⁷

The process involves four steps:

1. **Create ImageLabeler:** Instantiate the ImageLabeler class, optionally providing ImageLabelerOptions to configure parameters like a confidenceThreshold.
2. **Process Image:** Call the asynchronous processImage() method, passing in a prepared InputImage (which can be created from a File, bytes, or cameraImage).
3. **Parse Results:** Await the List<ImageLabel> and iterate over it to extract the label text and confidence score for each detected entity.
4. **Release Resources:** Call close() on the labeler instance to free up native resources.

Dart

```
// (Assuming 'inputImage' is an InputImage object)

// 1. Create the ImageLabeler instance
final options = ImageLabelerOptions(confidenceThreshold: 0.7);
final imageLabeler = ImageLabeler(options: options);

// 2. Process the image
final List<ImageLabel> labels = await imageLabeler.processImage(inputImage);

// 3. Iterate over the results
for (ImageLabel label in labels) {
  final String text = label.label;
  final double confidence = label.confidence;
  print('Found label: $text with confidence: $confidence');
}

// 4. IMPORTANT: Release resources
imageLabeler.close();
```

The New Wave: On-Device Generative AI (Gemini Nano)

On-device AI is evolving beyond these "classic" ML tasks (like classification and detection) and moving into generative AI. Google is now providing access to **Gemini Nano**, its smallest and most efficient foundation model, for on-device execution.¹⁸

This capability is being exposed through new **ML Kit GenAI APIs**.⁷ Rather than a raw prompt-in, text-out interface, these are high-level, use-case-specific APIs for tasks such as:

- Summarization
- Proofreading
- Rewriting (e.g., changing content style to 'formal' or 'add emojis')⁷

This technology operates within a new system service on Android called **AICore**, which leverages device-specific hardware (NPUs) for low-latency inference.¹⁸

This is, however, a bleeding-edge technology. As of mid-2024, Gemini Nano and AICore are only available on a limited set of high-end devices, such as the Google Pixel 8 Pro and Samsung S24 series.²¹ For this reason, while on-device generative AI is the clear future, most production-grade generative features today must rely on the cloud.

The Architectural Crossroads: On-Device vs. Cloud AI

This brings us to the central thesis of modern AI architecture. The choice between on-device and cloud AI is a spectrum of trade-offs.

- **On-device** offers unparalleled privacy, speed, and offline access. Its adoption, however, is constrained by the device's limited computational power, battery life, and storage.³ It is impossible to run a 100-billion parameter model on a smartphone.
- **Cloud AI** provides the opposite. It requires a stable internet connection and introduces data privacy considerations that must be managed. In exchange, it offers *near-unlimited* computational power.³ This allows for the use of massive, deep neural networks and transformer-based models.⁵
- A critical, and often overlooked, advantage of cloud AI is **model management**. A cloud-based model can be updated, patched, or hot-swapped *instantly* on the server. An on-device model, by contrast, must be pushed to users, often requiring a full application update, which can be hundreds of megabytes.³

The following table, synthesized from ³, summarizes this core architectural decision.

Feature	On-Device AI	Cloud AI
Privacy	High (data stays on device)	Lower (data is sent to servers)
Computational Power	Limited to device capabilities	Near-unlimited cloud resources
Internet Dependence	Works offline	Requires stable connection
Model Updates	Must be pushed to devices	Can be instantly deployed to cloud

The remainder of this lecture focuses on the "Cloud AI" column and its modern, secure implementation in Flutter.

Cloud-Based Generative AI: Firebase AI Logic

The Problem: The "Prototyping" Trap

When a Flutter developer first decides to use a cloud-based Gemini model, their initial research often leads them to the `Generative_ai` package.²² This package is an excellent, idiomatic Dart SDK that makes interacting with the Gemini API simple.²³

However, it contains a fatal flaw for production applications. To initialize the model, the developer must provide their secret API key directly in the client-side code²⁴, as seen in this

common example:

```
final model = GenerativeModel(model: 'MODEL_NAME', apiKey: apiKey);
```

This creates the **"Prototyping Trap"**: a secret API key has been embedded in a client application.²³ A malicious actor can easily decompile the app, extract this key, and use it to make calls on the developer's behalf, resulting in catastrophic billing.

This security vulnerability is so significant that the Flutter team themselves, who initially published the package, now explicitly recommend it for **prototyping only**.²³ This is not a viable path for a production-grade application.

The Solution: Firebase AI Logic

The secure, production-ready solution is **Firebase AI Logic**.²⁵

This service, formerly known as "Vertex AI in Firebase" ²³, is not merely a wrapper. It is a secure, serverless gateway that mediates all requests from the client application.

Here is how it solves the security flaw:

1. **No API Key:** The Flutter app *never* includes a secret API key.
2. **Firebase SDKs:** The client interacts with the service using the standard, client-side Firebase SDKs (available for Dart, Swift, Kotlin, etc.).²⁵
3. **App Check Integration:** Firebase AI Logic is built to integrate directly with **Firebase App Check**.²³ App Check verifies that incoming requests are originating from a legitimate, untampered instance of the application, and not from a malicious script or a "bot" that has stolen the app's identity.

This architecture shifts the security burden from the developer to the Firebase platform, providing the same level of protection for AI models that developers trust for Firestore and Firebase Authentication.

Setup & Configuration

Integrating Firebase AI Logic is a straightforward, four-step process:

1. **Firebase Console:** In the Firebase project console, the **Blaze (pay-as-you-go) plan** must be enabled. Then, the "Vertex AI API" must be enabled for the project.²⁸
2. **Add Dependencies:** The `firebase_ai` and `firebase_core` packages must be added to the project's `pubspec.yaml`.²⁹
3. **Configure App:** The FlutterFire CLI tool is used to connect the app to the Firebase project. Running `flutterfire configure` will automatically generate the `lib/firebase_options.dart` file.³⁰
4. **Initialize Firebase:** In `main.dart`, Firebase must be initialized before `runApp()` using the generated options file.²⁹

Bash

1. Console: Upgrade to Blaze Plan, Enable Vertex AI API

2. pubspec.yaml

```
flutter pub add firebase_ai
```

```
flutter pub add firebase_core
```

3. CLI

```
flutterfire configure
```

4. main.dart

```
import 'package:firebase_core/firebase_core.dart';
```

```
import 'firebase_options.dart';
```

```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await Firebase.initializeApp(  
    options: DefaultFirebaseOptions.currentPlatform,  
  );  
  runApp(const MyApp());  
}
```

Code Deep Dive: Text & Multi-modal Prompts

The resulting Dart code is clean and, most importantly, secure. The API signature is very similar to the prototyping package, but note the conspicuous absence of any API key.²³

The model is initialized using the `FirebaseVertexAI.instance`.

- **Text Prompts:** For a simple text-only query, a `Content.text()` object is passed to `generateContent()`.
- **Multi-modal Prompts:** A key strength of the Gemini models is their multi-modal capability.³³ To send a prompt containing both text and images, a `Content.multi()` constructor is used. This takes a list of `Part` objects, such as a `TextPart` for the text prompt and a `DataPart` for the image bytes.³⁴ The model can then reason about the text and image simultaneously.

Dart


```

import 'package:firebase_ai/firebase_ai.dart';
import 'dart:io'; // For File
import 'dart:typed_data'; // For Uint8List

// 1. Initialize the model (NO API KEY!)
final model = FirebaseVertexAI.instance.generativeModel(
  model: 'gemini-2.0-flash', // Use a recent model
);

// 2. Text-Only Prompt
final textPrompt =;
final response = await model.generateContent(textPrompt);
print(response.text);

// 3. Multi-modal Prompt (Text + Image)
// (Assuming 'selectedFile' is a File from an image_picker)
final Uint8List bytes = await selectedFile.readAsBytes();
final multiModalPrompt =)
];
final multiResponse = await model.generateContent(multiModalPrompt);
print(multiResponse.text);

```

Code Deep Dive: Streaming Responses

Waiting 10–30 seconds for a generative model to complete its response provides a poor user experience. For any chat-based application, streaming the response as it is generated is mandatory.³⁵

Firebase AI Logic makes this simple. Instead of calling `model.generateContent()`, the developer calls `model.generateContentStream()`.³³

This method returns a `Stream<GenerateContentResponse>`, which can be consumed using a standard `await for` loop.³⁶ This allows the UI to be updated in real-time as each "chunk" of text is received, token by token.

It is important to note that this is a *one-way* stream. For more advanced, low-latency, *bidirectional* conversations (e.g., a voice chat where a user can interrupt the model), Firebase AI Logic also provides a Live API that uses WebSockets and a `LiveSession` object.³⁷ For most text-based chat, however, `generateContentStream` is the correct and simpler solution.

Dart

```
// (Assuming 'model' and 'prompt' are defined)

// Call generateContentStream() instead of generateContent()
final Stream<GenerateContentResponse> responseStream =
    model.generateContentStream(prompt);

// Use an async 'await for' loop to listen
String fullResponse = '';
await for (final chunk in responseStream) {
    if (chunk.text!= null) {
        fullResponse += chunk.text!;
        // Update your UI here
        // e.g., myChatController.addText(chunk.text!);
    }
}
// Stream is finished. 'fullResponse' has the complete text.
```

Building Agents: Tool Calling in Flutter

Beyond Chat: What is Tool Calling?

The capabilities discussed so far enable building a powerful *chatbot*. The next and final leap is to build an *agent*.

An LLM, by itself, is a "brain in a jar." It is trained on a static, finite dataset. It does not know the current weather, it cannot access a private database, and it cannot interact with the application's UI or state.³¹

Tool Calling (or Function Calling) solves this. It gives the LLM "arms and legs".³⁹

The process is as follows:

1. The developer provides the LLM with a *description* (a schema) of functions (or "tools") that exist within the Dart application.
2. The LLM does **not** run this code.
3. Instead, when a user prompt requires external information, the LLM "pauses" its generation and returns a structured FunctionCall request, effectively saying, "Please execute this function for me with these specific arguments".⁴⁰
4. The Flutter app receives this request, executes its own Dart function, and sends the *result* back to the model.
5. The model then uses this new data to formulate its final, human-readable answer.

This is how an LLM can be connected to the "outside world," including real-time data and

application state.³¹

Client-Side Tool Calling: The Workflow

This "round-trip" ⁴¹ is the most critical concept to understand. The data flow for a user prompt like, "Make the app deep ocean blue," based on Google's "Colorist" Codelab ⁴², would be as follows:

1. **Step 1: Declare:** The Flutter app initializes the GenerativeModel with a Tool declaration for a Dart function named `setAppColor`.
2. **Step 2: Prompt:** The User sends the prompt: "Make the app deep ocean blue."
3. **Step 3: Model Request:** The Model's response is *not* text. It is a `FunctionCall` part, which is parsed as: Call '`setAppColor`' with args: `{"colorName": "deep ocean blue"}`.
4. **Step 4: App Execute:** The Dart code parses this request. It calls its *actual*, local `setAppColor("deep ocean blue")` function. This function might look up a hex code and update the app's `ThemeData` via a state manager.
5. **Step 5: App Respond:** The app sends the *result* of its execution (e.g., `{'hex': '#00429E'}`) back to the model in a new `Content.functionResponse`.
6. **Step 6: Model Synthesize:** The Model receives this new information and generates its final text response: "Okay, I've set the app theme to deep ocean blue, hex code #00429E."

Code Deep Dive: Client-Side Tool Calling

The following code demonstrates this workflow.

1. A Tool is defined with a `FunctionDeclaration`. This is just a schema to describe the function to the LLM.³⁹
2. The `GenerativeModel` is initialized *with this tool*.
3. The app sends the initial prompt.
4. The app checks the response for a `FunctionCall`.
5. If a `FunctionCall` is present, the app parses the name and args and executes its *actual* local Dart code.
6. The app sends the function's output back to the model using `Content.functionResponse`.
7. The app awaits the *final* response, which will now contain the synthesized, human-friendly text.

Dart

```
// 1. Define the tool (a description for the LLM)
```

```

final tool = Tool(functionDeclarations:),
)
]);

// 2. Initialize the model WITH the tool
final model = FirebaseVertexAI.instance.generativeModel(
  model: 'gemini-pro',
  tools: [tool],
);

// 3. Send the prompt
final response = await model.generateContent();

// 4. Check if the model wants to call our function
final functionCall = response.functionCall;
if (functionCall != null) {
  if (functionCall.name == 'findHexCode') {
    final colorName = functionCall.args['colorName'] as String;

    // 5. Execute YOUR actual Dart function
    final String hex = await myColorDatabase.getHexCode(colorName); // e.g., '#00429E'
    final responseJson = {'hex': hex, 'color': colorName};

    // 6. Send the function's result back to the model
    final finalResponse = await model.generateContent();

    // 7. Get the final, human-friendly text
    print(finalResponse.text); // "The hex code for deep ocean blue is #00429E."
  }
}

```

Advanced Orchestration: Genkit & MCPs

The Limits of Client-Side AI

Client-side tool calling is powerful, but it places the entire burden of orchestration logic within the Flutter application. This architecture breaks down when:

- **Complexity:** The AI's "tool" is not a single function call but a complex, 10-step business

process with its own logic.

- **Multi-Provider:** The developer wants to use a model from a *different* provider (e.g., Anthropic's Claude or OpenAI's GPT-4) for a specific task.²⁷
- **Advanced Workflows:** The task requires advanced techniques like Retrieval-Augmented Generation (RAG), which involves searching a vector database and feeding the results to the model.²⁷
- **Maintainability:** The developer wants to update the AI's logic, fix a bug in a prompt, or add a new tool *without* forcing all users to download a new version of the mobile app.⁴³

For these reasons, complex AI logic should be decoupled from the client. It should live on a server-side orchestrator. This is the purpose of **Firestore Genkit**.

Firestore AI Logic vs. Firestore Genkit

This is the final and most advanced architectural comparison.

- **Firestore AI Logic** (formerly Vertex AI in Firestore) is a **client-side SDK**. It is written in Dart, requires no backend, and is perfect for simple, secure, Google-only model calls directly from Flutter.²⁷
- **Firestore Genkit** is a **server-side, open-source framework**.⁴³ It is *not* a Flutter package. Genkit code is written in TypeScript/Node.js or Go.²⁷ It is designed to build, deploy, and monitor complex, multi-step "Flows".⁴⁶ It is multi-provider, supporting Google, OpenAI, Anthropic, and more.⁴³

The Flutter application's role in a Genkit architecture is to simply make a secure HTTP call (e.g., a Firestore Callable Function) to the Genkit backend, which handles all the complex AI orchestration.³¹

The following table summarizes this critical distinction.²⁷

Feature	Firestore AI Logic	Firestore Genkit
What it is	Client-side SDK (in Dart)	Server-side Framework (Node.js/Go)
Core Use Case	Simple, direct LLM calls from Flutter	Complex, multi-step "Flows"
Model Providers	Google-only (Gemini, Imagen)	Multi-provider (Google, OpenAI, Anthropic...)
Advanced Logic	Basic (Tool Calling)	Advanced (RAG, Caching, custom logic)
Flutter Dev	All in Dart	Flutter calls a backend (e.g., Cloud Function)

Deep Dive: Firestore Genkit

The Genkit architecture provides a powerful separation of concerns.³¹

1. **Flutter App:** The user provides trip details. The app makes a *single* call to a Firebase Callable Function, passing a JSON object. The app's only job is to send a request and display the final response.⁴⁸
2. **Genkit Backend:** This Firebase Function invokes a Genkit "Flow" (e.g., PackingHelperFlow).³¹
3. **The "Flow":** This TypeScript/Node.js code, running securely on a server, orchestrates the entire task.³¹ It might:
 - Call Tool 1 (e.g., a third-party Weather API).⁴⁹
 - Call Tool 2 (e.g., perform RAG on a vector database of packing items).
 - Call an LLM (from any provider) to synthesize this data into a checklist.
4. **Flutter App:** Receives the final, complete JSON response and renders it.

The power of this architecture is clear: the AI's "brain" (the flow) can be modified, debugged, and redeployed without ever touching the client-side Flutter code. Genkit also provides a "Dev UI" for testing and tracing these complex flows locally.⁴⁴

Understanding MCP (Model-Controller-Perception)

This brings us to the final, unifying concept: **MCP**. This is an architectural pattern for building robust agents.²⁶

- **Model:** This is the "brain," the core LLM (e.g., Gemini).
- **Perception:** These are the agent's "senses" and "limbs." This is the collection of **Tools** that the agent can use to interact with the world (APIs, databases, device functions).³¹
- **Controller:** This is the "brain stem" or the orchestration logic. It is the code that routes information, decides *which* tool to call, *when* to call it, and *how* to handle the response.

This pattern connects every topic covered in this lecture:

- **Firebase AI Logic's Tool Calling** (Module 5) is a simple, client-side implementation of the **Perception** (P) layer.
- **Firebase Genkit** (Module 6) is a full-featured, server-side framework for building the **Controller** (C) layer, allowing for the robust management of multiple Models (M) and Perceptions (P).

The Future: Agentic, Multi-modal Flutter Apps

The Future: Agentic Apps

The future of mobile applications lies not in the "app" but in the "agent." Google itself is "doubling down on agentic Flutter apps".⁴

The industry is moving beyond applications that users must *operate* and toward agents that perform tasks *on the user's behalf*.⁴ The travel app example³¹ is the archetype: a user does not *use* the app to find weather and then *use* it to find a packing list. The user states their *goal*—"I am going to Paris for 5 days"—and the *agent* orchestrates all necessary tools and logic to fulfill that goal. The app *acts* and *adapts*.⁴

What's Next?

As developers, there are three key trends to watch:

1. **Hyper-Personalization:** AI will move beyond powering discrete features to personalizing the entire user journey. It will analyze user behavior to proactively offer suggestions, dynamically re-rank UI elements, and predict user churn.⁴
2. **Truly Multi-Modal Interfaces:** The future is a seamless blend of voice, vision, and text.⁴ Users will interact with applications in the most natural way possible, and the application will be expected to understand and synthesize these varied inputs.³⁸
3. **AI-Assisted Development:** AI will not only power the applications; it will help build them. AI-powered pipelines are being developed to auto-generate Flutter and Firebase code directly from a developer's high-level idea, drastically shortening the distance from concept to deployment.²

In summary, Flutter made applications fast and beautiful. AI is now making them brilliant.²

Q&A and Final Resources

- **Flutter AI Toolkit:** flutter.dev/ai⁵⁵
- **Firebase AI Logic Docs:** firebase.google.com/docs/ai-logic²⁵
- **Firebase Genkit:** genkit.dev⁵⁶
- **Codelab (Agentic Apps):** codelabs.developers.google.com/flutter-gemini-colorist⁴²

Works cited

1. Getting Started with Flutter AI: A Complete Guide for App Development - Brilworks, accessed November 15, 2025, <https://www.brilworks.com/blog/getting-started-with-flutter-ai/>
2. Learn How AI Is Making Flutter Apps More Intelligent and User-Friendly in 2025, accessed November 15, 2025, <https://technorizen.com/learn-how-ai-is-making-flutter-apps-more-intelligent-and-user-friendly-in-2025/>
3. On-Device AI vs Cloud AI: Unlock Peak Performance for App, accessed

November 15, 2025,

<https://appbirds.co/on-device-ai-vs-cloud-ai-best-for-your-app/>

4. How Flutter and AI are Shaping the Future of App Development in 2025 - Medium, accessed November 15, 2025,
<https://medium.com/@flutternest/how-flutter-and-ai-are-shaping-the-future-of-app-development-in-2025-630f66a6adf6>
5. On-Device AI vs Cloud AI - What's Better for Tomorrow's Smart Infrastructure?, accessed November 15, 2025,
<https://astrikos.ai/on-device-ai-vs-cloud-ai-whats-better-for-tomorrows-smart-infrastructure/>
6. AI Integration in Flutter: Building Smarter Apps - Vibe Studio, accessed November 15, 2025,
<https://vibe-studio.ai/insights/ai-integration-in-flutter-building-smarter-apps>
7. ML Kit - Google for Developers, accessed November 15, 2025,
<https://developers.google.com/ml-kit>
8. Exploring the Power of Google ML Kit in Flutter | by Pranil Shah | Medium, accessed November 15, 2025,
<https://medium.com/@ipranilshah/exploring-the-power-of-google-ml-kit-in-flutter-d27a832be292>
9. google_ml_kit | Flutter package - Pub.dev, accessed November 15, 2025,
https://pub.dev/packages/google_ml_kit
10. Introducing ML Kit - Google for Developers Blog, accessed November 15, 2025,
<https://developers.googleblog.com/introducing-ml-kit/>
11. Google's ML Kit Object Detection and Tracking for Flutter - Pub.dev, accessed November 15, 2025,
https://pub.dev/documentation/google_mlkit_object_detection/latest/
12. google_mlkit_text_recognition | Flutter package - Pub.dev, accessed November 15, 2025, https://pub.dev/packages/google_mlkit_text_recognition
13. google_mlkit_document_scanner | Flutter package - Pub.dev, accessed November 15, 2025, https://pub.dev/packages/google_mlkit_document_scanner
14. Smart reply | ML Kit - Google for Developers, accessed November 15, 2025,
<https://developers.google.com/ml-kit/language/smart-reply>
15. google_mlkit_smart_reply | Flutter package - Pub.dev, accessed November 15, 2025, https://pub.dev/packages/google_mlkit_smart_reply
16. Smart Reply | ML Kit for Firebase - Google, accessed November 15, 2025,
<https://firebase.google.com/docs/ml-kit/generate-smart-replies>
17. google_mlkit_image_labeling example | Flutter package - Pub.dev, accessed November 15, 2025,
https://pub.dev/packages/google_mlkit_image_labeling/example
18. Gemini Nano | AI - Android Developers, accessed November 15, 2025,
<https://developer.android.com/ai/gemini-nano>
19. A walkthrough for Android's on-device GenAI solutions | Spotlight Week - YouTube, accessed November 15, 2025,
<https://www.youtube.com/watch?v=EpKghZYqVW4>
20. Gemini Nano on Android: Building with on-device gen AI - YouTube, accessed

- November 15, 2025, <https://www.youtube.com/watch?v=mP9QESmEDIs>
21. Gemini Nano: On-device AI solution : r/androiddev - Reddit, accessed November 15, 2025,
https://www.reddit.com/r/androiddev/comments/1ctz5je/gemini_nano_ondesign_solution/
 22. Harness the Gemini API in your Dart and Flutter Apps | by Ander Dobo, accessed November 15, 2025,
<https://blog.flutter.dev/harness-the-gemini-api-in-your-dart-and-flutter-apps-00573e560381>
 23. Using Firebase's Vertex AI for Flutter | by Sylvia Dieckmann - Medium, accessed November 15, 2025,
<https://medium.com/firebase-developers/using-firebases-vertex-ai-for-flutter-abdd85d1d1a8>
 24. Generative AI in Flutter: A Quickstart Guide - Stackademic, accessed November 15, 2025,
<https://blog.stackademic.com/google-generative-ai-in-flutter-a-quickstart-guide-3e5363c3521f>
 25. Gemini API using Firebase AI Logic - Google, accessed November 15, 2025,
<https://firebase.google.com/docs/ai-logic>
 26. Building AI-powered apps with Firebase AI Logic, accessed November 15, 2025,
<https://firebase.blog/posts/2025/05/building-ai-apps/>
 27. Choosing the Right AI Framework for Flutter: Firebase AI Logic vs ..., accessed November 15, 2025,
<https://itnext.io/choosing-the-right-ai-framework-for-flutter-firebase-ai-logic-vs-genkit-68888721efa7>
 28. Build Gen AI Apps Fast with Firebase and Flutter: A Hands-On Starter Guide - ITNEXT, accessed November 15, 2025,
<https://itnext.io/build-gen-ai-apps-fast-with-firebase-and-flutter-a-hands-on-starter-guide-54525e524e70>
 29. Using Vertex AI in Firebase SDKs for Flutter App | by Suesi Tran | Medium, accessed November 15, 2025,
<https://medium.com/@suesitrans/using-vertex-ai-in-firebase-sdks-for-flutter-app-e3dbc81fd044>
 30. The Future of Flutter GenAI is Here: Meet the firebase AI SDK | by Alfredo Bautista Santos, accessed November 15, 2025,
<https://alfredobs97.medium.com/the-future-of-flutter-genai-is-here-meet-the-firebase-ai-sdk-1ac5b4b22e9c>
 31. Building Agentic Apps - Flutter, accessed November 15, 2025,
<https://flutter.dev/events/building-agentic-apps>
 32. AI Toolkit - Flutter documentation, accessed November 15, 2025,
<https://docs.flutter.dev/ai-toolkit>
 33. Generate text using the Gemini API | Firebase AI Logic - Google, accessed November 15, 2025, <https://firebase.google.com/docs/ai-logic/generate-text>
 34. How to Build an AI-Powered Cooking Assistant with Flutter and Gemini, accessed November 15, 2025,

<https://www.freecodecamp.org/news/build-an-ai-powered-cooking-assistant-with-flutter-and-gemini/>

35. Stream generated text responses using the Gemini API | Firebase AI Logic - Google, accessed November 15, 2025, <https://firebase.google.com/docs/ai-logic/stream-responses>
36. How to implement Gemini into your Flutter app in 10 mins | by Apoorvpandey - Medium, accessed November 15, 2025, <https://apoorvpandey0.medium.com/how-to-implement-gemini-into-your-flutter-app-in-10-mins-4339c61a8da0>
37. Bidirectional streaming using the Gemini Live API | Firebase AI Logic - Google, accessed November 15, 2025, <https://firebase.google.com/docs/ai-logic/live-api>
38. Gemini Talk project: AI Live Multimodality in Flutter | by Alfredo ..., accessed November 15, 2025, <https://alfredobs97.medium.com/gemini-talk-project-ai-live-multimodality-in-flutter-ac37f1787709>
39. How to Build Agentic Apps with Flutter and Firebase AI Logic - Flutternest, accessed November 15, 2025, <https://flutternest.com/blog/agentic-apps-with-flutter>
40. Function calling using the Gemini API | Firebase AI Logic - Google, accessed November 15, 2025, <https://firebase.google.com/docs/ai-logic/function-calling>
41. Function calling with the Gemini API | Google AI for Developers, accessed November 15, 2025, <https://ai.google.dev/gemini-api/docs/function-calling>
42. Build a Gemini powered Flutter app - Google Codelabs, accessed November 15, 2025, <https://codelabs.developers.google.com/codelabs/flutter-gemini-colorist>
43. Building Agentic Apps with Flutter and Firebase Studio | Google Developer Groups, accessed November 15, 2025, <https://gdg.community.dev/events/details/google-gdg-mountain-view-presents-building-agentic-apps-with-flutter-and-firebase-studio/cohost-gdg-mountain-view>
44. Get started with Genkit - Firebase - Google, accessed November 15, 2025, <https://firebase.google.com/docs/genkit/get-started>
45. Getting Started with AI Image Generation Apps on Flutter, Genkit, and Imagen 3 - Medium, accessed November 15, 2025, <https://medium.com/@nozomi-koborinai/getting-started-with-ai-image-generation-apps-on-flutter-genkit-and-imagen-3-9a83c63cbdf3>
46. Master Generative AI with Genkit: Implementing Function Calling | Firebase Developers, accessed November 15, 2025, <https://medium.com/firebase-developers/implementing-function-calling-using-genkit-0c03f6cb9179>
47. firebase/genkit: Open-source framework for building AI-powered apps in JavaScript, Go, and Python, built and used in production by Google - GitHub, accessed November 15, 2025, <https://github.com/firebase/genkit>
48. Tool Calling with Genkit (With step-by-step code review) - YouTube, accessed November 15, 2025, <https://www.youtube.com/watch?v=d1E6TvrfEkg>
49. Giving your AI apps superpowers with Tool Calling - YouTube, accessed

- November 15, 2025, <https://www.youtube.com/watch?v=luZi09JfDo4>
50. How to build agentic apps with Flutter and Firebase AI Logic - YouTube, accessed November 15, 2025, https://www.youtube.com/watch?v=xo271p-Fl_4
51. Tool calling | Genkit - Firebase, accessed November 15, 2025, <https://firebase.google.com/docs/genkit/tool-calling>
52. How AI is Transforming Flutter App Development: Smarter, Faster, and More Scalable, accessed November 15, 2025, <https://solidappmaker.com/how-ai-is-transforming-flutter-app-development-smarter-faster-and-more-scalable/>
53. Build agentic apps with Flutter and Firebase AI Logic - Google IO, accessed November 15, 2025, <https://io.google/2025/explore/technical-session-6>
54. Flutter App Development in 2025: What an AI-Powered Pipeline Looks Like - Walturn, accessed November 15, 2025, <https://www.walturn.com/insights/flutter-app-development-in-2025-what-an-ai-powered-pipeline-looks-like>
55. AI - Flutter, accessed November 15, 2025, <https://flutter.dev/ai>
56. Genkit | Firebase - Google, accessed November 15, 2025, <https://firebase.google.com/docs/genkit/overview>